COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18//2026	**DATE SUBMITTED:**	09/28//2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 Tree ADT

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

Code

```
#include <iostream>
#include "GeneralTree.h"

int main(){
    // create the root node
    GeneralTree<char> tree('A');

    // create the first level
    tree.insert('A', 'B');
    tree.insert('A', 'C');
    tree.insert('A', 'D');
    tree.insert('A', 'E');
    tree.insert('A', 'F');
    tree.insert('A', 'G');

    // create the children of D
    tree.insert('D', 'H');

    // create the children of E
    tree.insert('E', 'I');
    tree.insert('E', 'J');
```

```
#ifndef GENERAL_TREE_H
#define GENERAL_TREE_H

template <typename T>
class GeneralTree{
    private:
        // create a key
        T key;
        // pointers for first child and next siblings
        GeneralTree<T>* firstChild;
        GeneralTree<T>* nextSibling;
        // search helper functions
        bool findPreOrder(T value);
        bool findInOrder(T value);
        bool findPostOrder(T value);
    public:
        // constructor
        GeneralTree(T value);
        // insert a child to a parent node
        void insert(T parent, T value);
        // traversing
        void preOrder();
        void inOrder();
        void postOrder();
        // searching
        void findData(int choice, T value);
};
```

```
// create the children of J
tree.insert('J', 'P');
tree.insert('J', 'Q');

// create the children of F
tree.insert('F', 'K');
tree.insert('F', 'L');
tree.insert('F', 'M');

// create the child of G
tree.insert('G', 'N');
```

```
template <typename T>
GeneralTree<T>::GeneralTree(T value){
    key = value;
    firstChild = nullptr;
    nextSibling = nullptr;
}
```

```
template <typename T>
void GeneralTree<T>::insert(T parent, T value){
    // if current node is parent
    if(key == parent){
        // add first child
        if(firstChild == nullptr){
            firstChild = new GeneralTree<T>(value);
        } else {
            // to last child
            GeneralTree<T>* current = firstChild;

            while(current->nextSibling != nullptr){
                current = current->nextSibling;
            }
            // add new child at the end
            current->nextSibling = new GeneralTree<T>(value);
        }
        return;
    }
    // find child, sibling
    if(firstChild != nullptr){
        firstChild->insert(parent, value);
    }

    if(nextSibling != nullptr){
        nextSibling->insert(parent, value);
    }
}
```

Observations	These code snippets contain one class, two core functions, and a driver code to initialize the general tree with the arrangement of values that follow the path of the provided general tree. The node 'A' is first initialized as a root node with six branches or children. This is followed by other branches of D, E, J, F, and G. which follow a similar process but with varying numbers of branches and different subtrees.

Table 9-1. General Tree Implementation.

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2

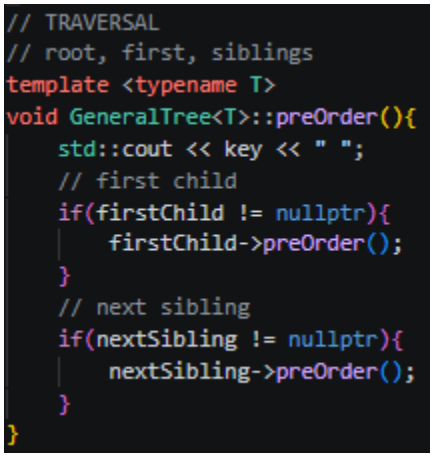
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-2. General Tree Node Index Table.

ILO B

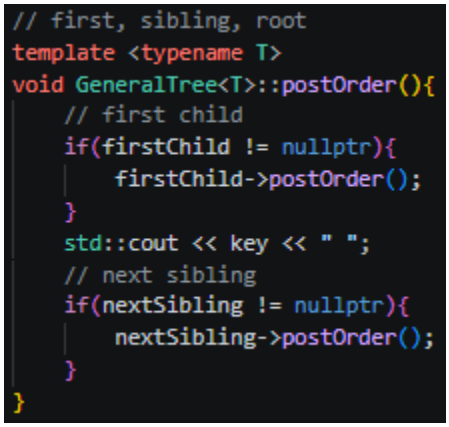
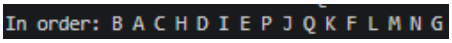
Pre-order	A -> B -> C -> D -> H -> E -> I -> J -> P -> Q -> F -> K -> L -> M -> G -> N
Post-order	B -> C -> H -> D -> I -> P -> Q -> J -> E -> K -> L -> M -> F -> N -> G -> A
In-order	B -> A -> C -> H -> D -> I -> E -> P -> J -> Q -> K -> F -> L -> M -> N -> G

Table 9-3. Manual Traversal Methods.

Function	Code	Output
pre-order		

```
// TRAVERSAL
// root, first, siblings
template <typename T>
void GeneralTree<T>::preOrder(){
    std::cout << key << " ";
    // first child
    if(firstChild != nullptr){
        firstChild->preOrder();
    }
    // next sibling
    if(nextSibling != nullptr){
        nextSibling->preOrder();
    }
}
```

```
Pre order: A B C D H E I J P Q F K L M G N
```

post-order		
in-order		

Observations:
The resulting outputs from the three recursive traversal functions match the results of the process done manually for each traversal path. Pre-order visits the parent before going to its children, while post-order visits the first child and siblings before the parent node. In-order visits the parent node right after the first child, and then proceeds with the remaining siblings.

Table 9-4. Traversal Methods.

Code

```
// Primary searching function
template <typename T>
void GeneralTree<T>::findData(int choice, T value){
    bool found = false;
    // choose a method
    switch(choice){
        case 1: found = findPreOrder(value); break;
        case 2: found = findInOrder(value); break;
        case 3: found = findPostOrder(value); break;
        default: break;
    }
    // display result
    if(found){
        std::cout << value << " was found! (Using method #" <<
        choice << ")\n";
    } else {
        std::cout << value << " was NOT found! (Using method #"
        << choice << ")\n";
    }
}
```

```
// Searching helper functions
template <typename T>
bool GeneralTree<T>::findPreOrder(T value){
    // check current node
    if(key == value){
        return true;
    }
    // first child
    if(firstChild != nullptr){
        if(firstChild->findPreOrder(value)){
            return true;
        }
    }
    // next sibling
    if(nextSibling != nullptr){
        if(nextSibling->findPreOrder(value)){
            return true;
        }
    }
    return false;
}
```

```
template <typename T>
bool GeneralTree<T>::findInOrder(T value){
    // first child
    if(firstChild != nullptr){
        if(firstChild->findInOrder(value)){
            return true;
        }
    }
    // check current node
    if(key == value){
        return true;
    }
    // next sibling
    if(nextSibling != nullptr){
        if(nextSibling->findInOrder(value)){
            return true;
        }
    }
    return false;
}
```

```
template <typename T>
bool GeneralTree<T>::findPostOrder(T value){
    // first child
    if(firstChild != nullptr){
        if(firstChild->findPostOrder(value)){
            return true;
        }
    }
    // next sibling
    if(nextSibling != nullptr){
        if(nextSibling->findPostOrder(value)){
            return true;
        }
    }
    // check current node
    if(key == value){
        return true;
    }
    return false;
}
```

```
std::cout << "\n\nfindData: 1 = pre-order, 2 =
in-order, 3 = post-order";
std::cout << "\nFinding char 'N'...\n";
tree.findData(1, 'N');
tree.findData(2, 'N');
tree.findData(3, 'N');
```

```
std::cout << "\nFinding char 'i'...\n";
tree.findData(1, 'i');
tree.findData(2, 'i');
tree.findData(3, 'i');
```

Output

```
findData: 1 = pre-order, 2 = in-order, 3 = post-order
Finding char 'N'...
N was found! (Using method #1)
N was found! (Using method #2)
N was found! (Using method #3)
```

```
Finding char 'i'...
i was NOT found! (Using method #1)
i was NOT found! (Using method #2)
i was NOT found! (Using method #3)
```

Observations

Each traversal method used in finding a certain node of the tree results in the exact same output. The only notable difference is how each searching method traverses through the tree to find the assigned node, and which one finds the node first. In this case, post-order traversal reaches N first after visiting the subtrees of B, C, D, E, and F. As specified in the instructions, the function used has a parameter for selecting the traversal method and a node to find.

Table 9-5. Find Data Function.

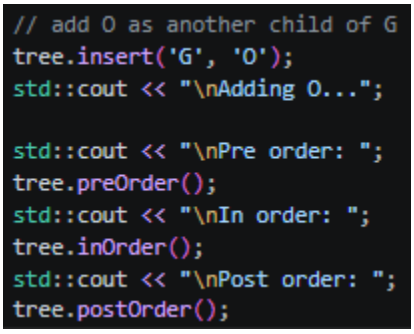
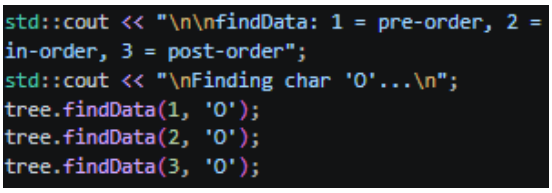
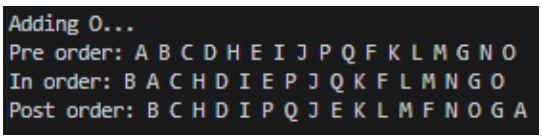
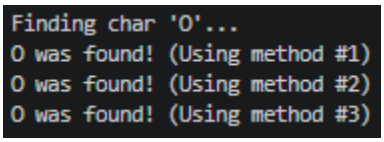
Code & Output	   
Observations	Char node 'O' is inserted into the subtree of node G with the same procedure done in Table 9-1. This addition results in two children for node G, where O is now the sibling of child N. Looking at the traversal outputs, O shows up next to N or G depending on the traversal method. The search results all result with O being found regardless of which traversal method is used.

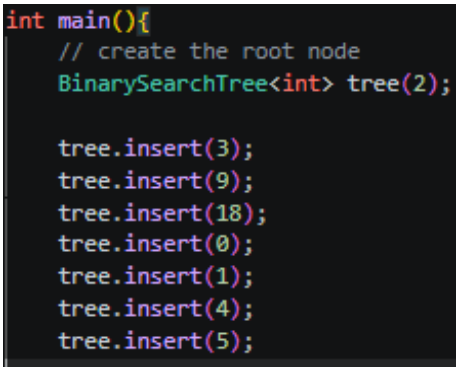
Table 9-6. Leaf Node.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

Main	

Header

```
template <typename T>
class BinarySearchTree{
    private:
        // create a key
        T key;
        // pointers for left and right
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;
    public:
        // constructor
        BinarySearchTree(T value);

        // insert left and right
        void insert(T value);
        // traversing
        void preOrder();
        void inOrder();
        void postOrder();
        // searching
};
```

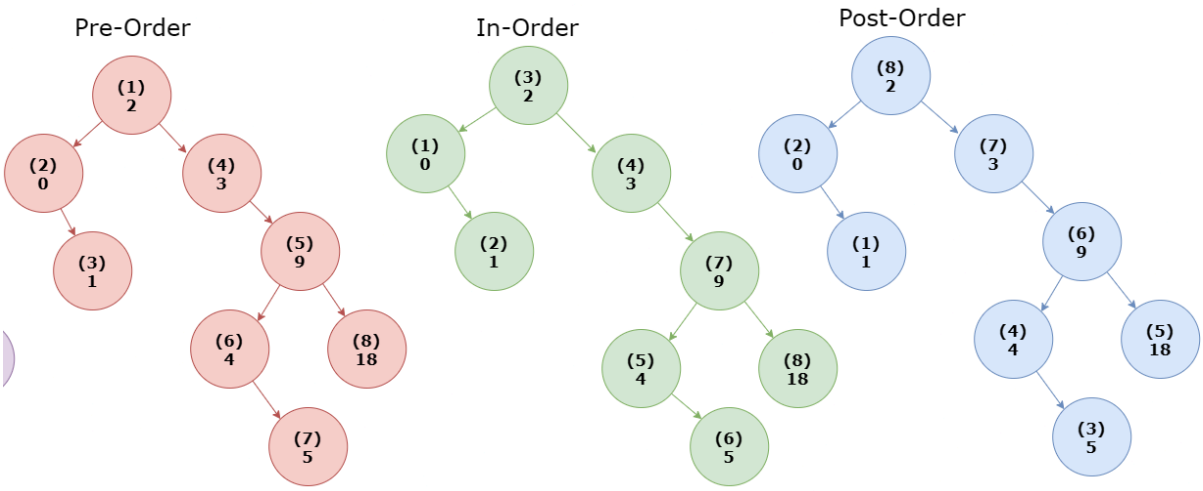
```
template <typename T>
void BinarySearchTree<T>::insert(T value){
    // left or right

    // go left if value < key insert to left
    if(value < key){
        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        } else {
            left->insert(value);
        }
    } else if(value > key){
        if(right == nullptr){
            right = new BinarySearchTree<T>(value);
        } else {
            right->insert(value);
        }
    }
}
```

```
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}
```

Observations:
This binary search tree implementation inserts the specified list of numbers to the binary tree. Compared to the general tree, the insertion function does not require a parameter that explicitly takes the parent and child node. Instead, the insertion of nodes into the subtrees only has two valid places, left and right. For instance, if the value is less than the key, then traverse to the left and insert the assigned value from the parameter. The insertion function is also simpler since there are only two places to traverse and insert, unlike general trees where the number of children or siblings varies.

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.



Note: The sequence of traversal is denoted by the numbers in parentheses.

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

Function	Main/Output	Observations

`template <typename T>`
`void BinarySearchTree<T>::preOrder(){`
`  std::cout<< key << " ";`
`  if (left != nullptr){`
`  left->preOrder();`
`  }`
`  if (right != nullptr){`
`  right->preOrder();`
`  }`
`}`
	`std::cout << "Pre order: ";`
	`tree.preOrder();`
	`Pre order: 2 0 1 3 9 4 5 18`
		This recursive pre-order traversal method follows the ROOT->LEFT->RIGHT pattern, where the traversal visits the root nodes first, then the nodes at the left subtree, and then the nodes at the right subtree.

`// aggressively go to the left then right`
`// left root right`
`template <typename T>`
`void BinarySearchTree<T>::inOrder(){`
`  // go to left`
`  if (left != nullptr){`
`  left->inOrder();`
`  }`
`  std::cout<< key << " ";`
`  // goto right`
`  if (right != nullptr){`
`  right->inOrder();`
`  }`
`}`
	`std::cout << "\nIn order: ";`
	`tree.inOrder();`
	`In order: 0 1 2 3 4 5 9 18`
		This recursive in-order traversal method follows the LEFT->ROOT->RIGHT pattern, where the traversal visits the nodes at the left subtrees, then the root nodes, and then the nodes at the right subtrees.

```
// left then right then root
template <typename T>
void BinarySearchTree<T>::postOrder(){
    // go to left
    if (left != nullptr){
        left->postOrder();
    }
    // goto right
    if (right != nullptr){
        right->postOrder();
    }
    std::cout<< key << " ";
}
```

```
std::cout << "\nPost order: ";
tree.postOrder();
```

```
Post order: 1 0 5 4 18 9 3 2
```

This recursive in-order traversal method follows the LEFT->RIGHT->ROOT pattern, where the traversal visits the nodes at the left subtrees, then the nodes at the right subtrees, and then the root nodes.

Notice how the print key instruction and if statements with the left and right variables are arranged in the code.

Is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2?

- There is no difference in the output of the traversal methods generated by the program in the code implementation of each algorithm performing the three traversal methods. The code outputs match the indicated diagrams shown in item #2. Each function correctly follows their specified traversal sequence.

Conclusion

This hands-on activity has demonstrated the concepts of General Trees and Binary Trees along with three mentioned traversal methods and modifications to add searching functionalities. The three traversal methods, pre-order, in-order, and post-order traversals, all follow a similar three-step process to traverse through the list but with different arrangements. Generally, the traversal involves visiting the nodes of the parent, child, and its siblings.

The procedures introduced the general tree and its parts in the code, such as the class containing the variables, tree and insertion functions, and the driver code that inserts the characters into the tree. The index table contains the placements of the nodes in the tree according to their height and depth based on the direction of measurement and number of edges. A zero is determined in depth measurement when it is a root node, while it is determined for height measurement when it is a leaf node. Additionally, the next part compares manual and algorithmic traversals with three code implementations and outputs, and these were modified to find a certain node. The supplementary activities feature an alternative to the general tree called the binary search tree, which is simpler in design and implementation due to the number of children of the parent nodes being fixed to just two. A similar process is done for this tree by including the general code structure containing the classes, functions, and driver code to input the data into the nodes, and implementing three traversal methods.

Overall, this activity has introduced a new part of non-linear data structures by demonstrating the general and binary search tree with their respective algorithms, implementations, and comparisons to manual traversal methods. The underlying process in these trees are illustrated clearly with the provided diagrams and helped one design their own diagrams for their own data trees, such as the figure shown in ILO C that illustrates the steps of each traversal method.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation and clarification of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain the given concepts discussed in the activity and clarify the more abstract concepts in the procedures.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.